Approximations

April 14, 2026

1 Approximations

1.1 Contents

1.2 Notes

- The notebooks are largely self-contained. If a symbol appears, an explanation will be present at some point in the same notebook — most often with a link to the cell where it is explained.
- **GitHub does a poor job of rendering these notebooks.** The online render is missing links, symbols are absent, and notations are badly formatted. Clone a local copy or download the notebook and open it locally, or refer to the PDFs in the `PDFs/` folder.
- All code uses only Python's standard library.
- Approximation notation appears frequently in algorithm analysis, numerical methods, and asymptotic expansions. The symbols in this notebook describe the relative behavior of functions as their arguments grow large or tend to zero.

See the Functions notebook for familiarity with function notation used throughout this notebook.

1.3 Importing Libraries

[1]:
```
import math
```

Approximate Equality

The simplest and most common notation for expressing that two quantities are approximately equal is $\approx$, denoted by:

$$a \approx b$$

For example, $\pi \approx 3.14$. Other symbols occasionally used for approximate equality include $\doteq$, $\cong$, $\sim$, and $\simeq$, though $\approx$ is the most widely adopted.

[2]:
```python
x = math.pi
approx = 3.14
print(f'pi       = {x}')
print(f'approx   = {approx}')
print(f'|pi - approx| = {abs(x - approx):.6f}')
```

```
pi       = 3.141592653589793
approx   = 3.14
|pi - approx| = 0.001593
```

Plus-Minus Tolerance

A more precise way to express approximate equality is to state a tolerance using the plus-minus symbol $\pm$. The expression $a = b \pm \varepsilon$ is shorthand for:

$$b - \varepsilon \leq a \leq b + \varepsilon$$

For example, $\pi = 3.14 \pm 0.002$ asserts that $3.138 \leq \pi \leq 3.142$.

[3]:
```python
x = math.pi
center, tol = 3.14, 0.002
lower, upper = center - tol, center + tol
print(f'{lower} <= pi <= {upper}')
print(f'{lower} <= {x:.6f} <= {upper}')
print('Holds:', lower <= x <= upper)
```

```
3.1380000000000003 <= pi <= 3.142
3.1380000000000003 <= 3.141593 <= 3.142
Holds: True
```

For more information on limits, see the Calculus notebook.

Asymptotic To

The symbol $\sim$ between two functions means *is asymptotic to*. It asserts that the limit of the ratio of the two expressions approaches 1. Given functions f and g of x:

- When x is small, $f(x) \sim g(x)$ means:

$$\lim_{x\to 0} \frac{f(x)}{g(x)} = 1$$

- When x is large, $f(x) \sim g(x)$ means:

$$\lim_{x\to \infty} \frac{f(x)}{g(x)} = 1$$

For example, when x is small, $e^x \sim 1 + x + \frac{1}{2}x^2$. When n is large, $\binom{n}{3} \sim \frac{n^3}{6}$.

[4]:
```
# e^x ~ 1 + x + x^2/2 for small x
for x in [1.0, 0.1, 0.01, 0.001]:
    f = math.exp(x)
    g = 1 + x + 0.5 * x**2
    print(f'x={x:<6}  e^x / (1 + x + x^2/2) = {f/g:.10f}')
```

```
x=1.0     e^x / (1 + x + x^2/2) = 1.0873127314
x=0.1     e^x / (1 + x + x^2/2) = 1.0001546770
x=0.01    e^x / (1 + x + x^2/2) = 1.0000001654
x=0.001   e^x / (1 + x + x^2/2) = 1.0000000002
```

[5]:
```
# C(n,3) ~ n^3/6 for large n
for n in [10, 100, 1000, 10000]:
    f = math.comb(n, 3)
    g = n**3 / 6
    print(f'n={n:<6}  C(n,3) / (n^3/6) = {f/g:.10f}')
```

```
n=10      C(n,3) / (n^3/6) = 0.7200000000
n=100     C(n,3) / (n^3/6) = 0.9702000000
n=1000    C(n,3) / (n^3/6) = 0.9970020000
n=10000   C(n,3) / (n^3/6) = 0.9997000200
```

Proportional To

The expression $f(x) \propto g(x)$ indicates that the two functions are *proportional* to each other. In the strict sense, this means there exists a positive constant k such that:

$$f(x) = k\,g(x)$$

The symbol $\propto$ is also used in an approximate sense in which lower-order terms are neglected. In this case, $f(x) \propto g(x)$ means that $f(x)/g(x)$ tends to a finite, nonzero limit as $x \to \infty$.

For example, $\binom{n}{3} \propto n^3$ because $\binom{n}{3}/n^3 \to \frac{1}{6}$ as $n \to \infty$.

[6]:
```
# strict proportionality: f(x) = 3 * g(x)
f = lambda x: 3 * x**2
g = lambda x: x**2
for x in [1, 5, 10, 100]:
```

```
    print(f'x={x:<4}  f(x)/g(x) = {f(x)/g(x):.1f}')
```

```
x=1     f(x)/g(x) = 3.0
x=5     f(x)/g(x) = 3.0
x=10    f(x)/g(x) = 3.0
x=100   f(x)/g(x) = 3.0
```

[7]:
```
# approximate: C(n,3) / n^3 -> 1/6
for n in [10, 100, 1000, 10000]:
    ratio = math.comb(n, 3) / n**3
    print(f'n={n:<6}  C(n,3)/n^3 = {ratio:.6f}  (-> 1/6)')
```

```
n=10      C(n,3)/n^3 = 0.120000  (-> 1/6)
n=100     C(n,3)/n^3 = 0.161700  (-> 1/6)
n=1000    C(n,3)/n^3 = 0.166167  (-> 1/6)
n=10000   C(n,3)/n^3 = 0.166617  (-> 1/6)
```

Comparable To

The symbol $\asymp$ expresses a similar but less restrictive relationship than $\sim$ or $\propto$. The ratio between the two expressions need not tend to a limit; all that is required is that the ratio be bounded away from 0 and ∞.

Given $f, g : \mathbb{R} \to \mathbb{R}_+$, the expression $f \asymp g$ means there exist positive numbers t, a, and b such that:

$$\exists\, t, a, b > 0, \quad \forall\, x \geq t, \quad a \leq \frac{f(x)}{g(x)} \leq b$$

In some cases, $f(x) \asymp g(x)$ may be applied to functions that return negative values. In that case, $a \leq \left|\frac{f(x)}{g(x)}\right| \leq b$ when x is sufficiently large.

[8]:
```
# (x^2 + 3x) is comparable to x^2 for large x
for x in [10, 100, 1000, 10000]:
    f = x**2 + 3 * x
    g = x**2
    print(f'x={x:<6}  (x^2 + 3x)/x^2 = {f/g:.6f}')
print('\nBounded: 1.0 <= ratio <= 1.3 for x >= 10')
```

```
x=10      (x^2 + 3x)/x^2 = 1.300000
x=100     (x^2 + 3x)/x^2 = 1.030000
x=1000    (x^2 + 3x)/x^2 = 1.003000
x=10000   (x^2 + 3x)/x^2 = 1.000300

Bounded: 1.0 <= ratio <= 1.3 for x >= 10
```

Big O

The big-oh notation of Landau is a useful shorthand for giving an approximate bound for a function or for indicating relatively unimportant terms in an expression. The notation $f(x) = O(g(x))$ is an implicit *upper bound* on $|f(x)|$. It asserts that $|f(x)| \le b\,|g(x)|$ where b is some positive number.

- In the case of x near 0, the notation $f(x) = O(g(x))$ means there are positive numbers ε and b such that for all x between $-\varepsilon$ and ε:

$$\left|\frac{f(x)}{g(x)}\right| \le b$$

- In the case of large x, the notation $f(x) = O(g(x))$ means there are positive numbers t and b so that for all $x \ge t$:

$$\left|\frac{f(x)}{g(x)}\right| \le b$$

For example, $\binom{n}{3} = \frac{1}{6}n^3 - \frac{1}{2}n^2 + \frac{1}{3}n$, so $\binom{n}{3} = O(n^3)$. For x near 0, $e^x = 1 + x + O(x^2)$.

The $=$ sign in big-oh expressions is mildly illogical. It is more proper to write $\binom{n}{3} \in O(n^3)$, as $O(n^3)$ denotes the set of all functions that are asymptotically bounded by n^3. Some authors use a calligraphic $\mathcal{O}$ for the big-oh notation.

[9]:
```
# e^x = 1 + x + O(x^2) near 0
# |e^x - 1 - x| / x^2 is bounded
for x in [0.5, 0.1, 0.01, 0.001]:
    remainder = abs(math.exp(x) - 1 - x)
    print(f'x={x:<6}  |e^x - 1 - x|/x^2 = {remainder / x**2:.6f}')
```

```
x=0.5     |e^x - 1 - x|/x^2 = 0.594885
x=0.1     |e^x - 1 - x|/x^2 = 0.517092
x=0.01    |e^x - 1 - x|/x^2 = 0.501671
x=0.001   |e^x - 1 - x|/x^2 = 0.500167
```

[10]:
```
# C(n,3) = O(n^3) for large n
# C(n,3) / n^3 is bounded above
for n in [10, 100, 1000, 10000]:
    ratio = math.comb(n, 3) / n**3
    print(f'n={n:<6}  C(n,3)/n^3 = {ratio:.6f}  (<= 0.167)')
```

```
n=10      C(n,3)/n^3 = 0.120000  (<= 0.167)
n=100     C(n,3)/n^3 = 0.161700  (<= 0.167)
n=1000    C(n,3)/n^3 = 0.166167  (<= 0.167)
n=10000   C(n,3)/n^3 = 0.166617  (<= 0.167)
```

Big Omega

The notation $f(x) = \Omega(g(x))$ is an implicit *lower bound* on $|f(x)|$. The precise meaning depends on whether small or large values of x are considered.

- In the case of x near 0, the notation $f(x) = \Omega(g(x))$ means there are positive numbers ε and b such that for all x between $-\varepsilon$ and ε:

$$\left|\frac{f(x)}{g(x)}\right| \geq b$$

- In the case of large x, the notation $f(x) = \Omega(g(x))$ means there are positive numbers t and b so that for all $x \geq t$:

$$\left|\frac{f(x)}{g(x)}\right| \geq b$$

Because Ω represents a lower bound, it is correct to write $\binom{n}{3} = \Omega(n^3)$ but it is incorrect to write $\binom{n}{3} = \Omega(n^k)$ for any $k \geq 4$.

[11]:

```
# C(n,3) = Omega(n^3): ratio is bounded below
for n in [10, 100, 1000, 10000]:
    ratio = math.comb(n, 3) / n**3
    print(f'n={n:<6}  C(n,3)/n^3 = {ratio:.6f}  (>= 0.10)')
```

```
n=10      C(n,3)/n^3 = 0.120000  (>= 0.10)
n=100     C(n,3)/n^3 = 0.161700  (>= 0.10)
n=1000    C(n,3)/n^3 = 0.166167  (>= 0.10)
n=10000   C(n,3)/n^3 = 0.166617  (>= 0.10)
```

Big Theta

The notation $f(x) = \Theta(g(x))$ means that both $f(x) = O(g(x))$ and $f(x) = \Omega(g(x))$ hold. That is, there are positive constants a and b such that for all sufficiently large (or small) x:

$$a \leq \left|\frac{f(x)}{g(x)}\right| \leq b$$

Note that:

$$f(x) = \Theta(g(x)) \iff f(x) \asymp g(x) \iff g(x) = \Theta(f(x))$$

(See: Comparable To)

It is correct to write $\binom{n}{3} = \Theta(n^3)$ but it is incorrect to write $\binom{n}{3} = \Theta(n^k)$ for any $k \neq 3$.

[12]:

```
# C(n,3) = Theta(n^3): bounded above AND below
for n in [10, 100, 1000, 10000]:
    ratio = math.comb(n, 3) / n**3
    print(f'n={n:<6}  C(n,3)/n^3 = {ratio:.6f}')
print('\nBounded: 0.10 <= ratio <= 0.17 for all n >= 10')
```

```
n=10      C(n,3)/n^3 = 0.120000
n=100     C(n,3)/n^3 = 0.161700
n=1000    C(n,3)/n^3 = 0.166167
n=10000   C(n,3)/n^3 = 0.166617

Bounded: 0.10 <= ratio <= 0.17 for all n >= 10
```

Little o

The notation $f(x) = o(g(x))$ indicates that $f(x)$ is *much smaller* than $g(x)$ in the sense that their ratio tends to zero. The precise meaning depends on context: small x or large x. In the appropriate case, $f(x) = o(g(x))$ means one of the following:

$$\lim_{x\to 0} \frac{f(x)}{g(x)} = 0 \qquad \text{or} \qquad \lim_{x\to\infty} \frac{f(x)}{g(x)} = 0$$

For example, when x is small, $\cos x = 1 - \frac{1}{2}x^2 + o(x^3)$. When n is large, $n! = o(n^n)$.

A $o(1)$ term in an expression stands for a quantity that tends to 0.

[13]:
```python
# cos x = 1 - x^2/2 + o(x^3) for small x
# |cos x - 1 + x^2/2| / |x^3| -> 0
for x in [1.0, 0.1, 0.01, 0.001]:
    remainder = abs(math.cos(x) - 1 + 0.5 * x**2)
    print(f'x={x:<6}  |cos x - 1 + x^2/2|/|x^3| = {remainder / abs(x**3):.8f}')
```

```
x=1.0     |cos x - 1 + x^2/2|/|x^3| = 0.04030231
x=0.1     |cos x - 1 + x^2/2|/|x^3| = 0.00416528
x=0.01    |cos x - 1 + x^2/2|/|x^3| = 0.00041667
x=0.001   |cos x - 1 + x^2/2|/|x^3| = 0.00004167
```

[14]:
```python
# n! = o(n^n) for large n: ratio tends to 0
for n in [5, 10, 15, 20]:
    ratio = math.factorial(n) / n**n
    print(f'n={n:<3}  n!/n^n = {ratio:.10f}')
```

```
n=5    n!/n^n = 0.0384000000
n=10   n!/n^n = 0.0003628800
n=15   n!/n^n = 0.0000029863
n=20   n!/n^n = 0.0000000232
```

Little omega

The notation $f(x) = \omega(g(x))$ has the opposite meaning of little-o: $f(x)$ is *much larger* than $g(x)$. Concisely:

$$f(x) = \omega(g(x)) \iff g(x) = o(f(x))$$

In other words, $f(x) = \omega(g(x))$ means one of the following, depending on context:

$$\lim_{x \to 0} \left| \frac{f(x)}{g(x)} \right| = \infty \qquad \text{or} \qquad \lim_{x \to \infty} \left| \frac{f(x)}{g(x)} \right| = \infty$$

In an expression, $\omega(1)$ represents a term that is tending to infinity.

[15]:
```
# n^n = omega(2^n) for large n: ratio tends to infinity
for n in [5, 10, 15, 20]:
    ratio = n**n / 2**n
    print(f'n={n:<3}  n^n/2^n = {ratio:.2f}')
```

```
n=5    n^n/2^n = 97.66
n=10   n^n/2^n = 9765625.00
n=15   n^n/2^n = 13363461010158.06
n=20   n^n/2^n = 100000000000000000000.00
```

Much Less Than and Much Greater Than

Alternative ways to express $f(x) = o(g(x))$ and $f(x) = \omega(g(x))$ are, respectively, $f(x) \ll g(x)$ and $f(x) \gg g(x)$. The relation $\ll$ expresses the notion of *much less than* and $\gg$ indicates *much greater than*.

For example, for large n:

$$n^2 \ll 2^n \qquad \text{because} \qquad \lim_{n \to \infty} \frac{n^2}{2^n} = 0$$

$$n^n \gg n! \qquad \text{because} \qquad \lim_{n \to \infty} \frac{n^n}{n!} = \infty$$

[16]:
```
# n^2 << 2^n: ratio tends to 0 for large n
for n in [5, 10, 20, 30]:
    ratio = n**2 / 2**n
    print(f'n={n:<3}  n^2/2^n = {ratio:.10f}')
```

```
n=5    n^2/2^n = 0.7812500000
n=10   n^2/2^n = 0.0976562500
n=20   n^2/2^n = 0.0003814697
n=30   n^2/2^n = 0.0000008382
```

[17]:
```
# n^n >> n!: ratio tends to infinity for large n
for n in [5, 10, 15, 20]:
    ratio = n**n / math.factorial(n)
    print(f'n={n:<3}  n^n/n! = {ratio:.2f}')
```

```
n=5    n^n/n! = 26.04
n=10   n^n/n! = 2755.73
```

```
n=15   n^n/n! = 334864.63
n=20   n^n/n! = 43099804.12
```